

2. REVIEW & ANALYZE REQUIREMENTS

Before designing and executing any testing activities, it is important to review and analyze the project requirements. Requirement analysis helps identify the functionality to be tested, security objectives, project boundaries, and potential risks. For this project, the analysis focuses on the security testing of the Damn Vulnerable Web Application (DVWA) using Selenium WebDriver, TestNG, Maven, and OWASP ZAP.

2.1 Functional Requirements

Functional requirements define the features and operations that the application must perform correctly. These requirements describe how users interact with the system and what results should be produced.

For this project, the following functional requirements were identified:

User Authentication

The application must allow users to log in using valid credentials. After successful authentication, users should be redirected to the DVWA home page.

The system should:

- Accept valid username and password combinations.
- Reject invalid login attempts.
- Display appropriate error messages for invalid credentials.
- Maintain user sessions after successful login.
- Allow users to access authorized application modules.

Navigation Functionality

The application should provide access to different vulnerability modules after login.

Users should be able to:

- Navigate to the SQL Injection module.
- Navigate to the XSS module.
- Access other security testing pages.
- Move between application pages without errors.

SQL Injection Module

The application should process user input submitted through SQL Injection forms.

Expected behavior includes:

- Accepting valid user input.
- Returning corresponding records.
- Handling malicious SQL payloads appropriately.
- Preventing unauthorized database access.

Cross-Site Scripting (XSS) Module

The application should process and display user input.

Expected behavior includes:

- Displaying valid user data.
- Preventing execution of malicious scripts.
- Sanitizing potentially dangerous input.
- Protecting users from client-side attacks.

Report Generation

The automation framework should generate execution reports after test completion.

The framework should:

- Generate TestNG reports.
 - Generate Extent Reports.
 - Capture execution status.
 - Store screenshots when required.
 - Provide execution summaries.
-

2.2 Security Requirements

Security requirements define the protection mechanisms that should be implemented to safeguard the application against vulnerabilities and attacks.

The primary goal of this project is to verify whether the application satisfies these security requirements.

Authentication Security

The application should protect user accounts from unauthorized access.

Security controls should include:

- Secure login validation.
- Session management.
- Access control mechanisms.
- Protection against credential manipulation.

Input Validation

All user-supplied data should be validated before processing.

The application should:

- Reject malicious inputs.
- Prevent injection attacks.
- Filter dangerous characters.
- Validate form data before submission.

SQL Injection Protection

The application should prevent attackers from manipulating SQL queries.

Security mechanisms should include:

- Parameterized queries.
- Prepared statements.
- Input validation.
- Secure database communication.

The application should not allow attackers to:

- Access unauthorized records.
- Modify database contents.
- Bypass authentication.
- Execute arbitrary SQL commands.

Cross-Site Scripting (XSS) Protection

The application should prevent execution of malicious scripts.

Protection mechanisms should include:

- Input sanitization.
- Output encoding.
- Content Security Policies.
- Secure rendering of user data.

The application should prevent:

- Script execution.

- Session theft.
- Cookie manipulation.
- Browser-based attacks.

Security Header Implementation

The application should implement secure HTTP headers such as:

- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options
- Strict-Transport-Security
- Referrer-Policy

These headers help reduce common web application security risks.

Vulnerability Detection

The system should support security scanning activities using OWASP ZAP.

The scanning process should:

- Discover application pages.
- Identify vulnerabilities.
- Classify risk levels.
- Generate security reports.

2.3 Test Scope

The test scope defines the boundaries of testing activities and identifies what will and will not be tested during project execution.

In Scope

The following areas are included within the scope of testing:

Login Module Testing

Testing activities include:

- Login verification.
- Authentication validation.
- Navigation after login.
- Session establishment.

SQL Injection Testing

Testing activities include:

- Execution of SQL payloads.
- Vulnerability identification.
- Database response validation.
- Security weakness analysis.

XSS Testing

Testing activities include:

- Execution of JavaScript payloads.
- Detection of reflected XSS vulnerabilities.
- Input validation testing.
- Security assessment.

OWASP ZAP Testing

Testing activities include:

- Spider Scan execution.
- Active Scan execution.
- Vulnerability identification.
- Security report generation.

Automation Framework Validation

Testing activities include:

- Selenium automation execution.
- TestNG execution.
- Maven build verification.
- Report generation validation.

Reporting Validation

Testing activities include:

- TestNG report verification.
- Extent Report verification.
- OWASP ZAP report verification.

Out of Scope

The following activities are excluded from this project:

- Real-world penetration testing.
- Production environment testing.
- Denial-of-Service attacks.
- Mobile application security testing.
- API security testing.
- Network security assessment.
- Cloud security testing.
- Performance testing.
- Load testing.

The project is restricted to web application security testing within a controlled local environment.

2.4 Risk Analysis

Risk analysis identifies potential issues that may affect project execution, test accuracy, or overall project success.

Technical Risks

Tool Integration Failure

There is a possibility that Selenium, TestNG, Maven, and OWASP ZAP may not integrate correctly.

Impact:

- Test execution failures.
- Security scans may not run successfully.
- Report generation issues.

Mitigation:

- Verify tool compatibility.
 - Perform integration testing before execution.
-

OWASP ZAP Connectivity Issues

OWASP ZAP may fail to establish communication with the automation framework.

Impact:

- Spider Scan failure.
- Active Scan failure.

- Incomplete security reports.

Mitigation:

- Verify proxy configuration.
 - Validate API connectivity before execution.
-

Environment Configuration Issues

Incorrect setup of XAMPP, DVWA, MySQL, Java, or browser drivers may cause execution failures.

Impact:

- Application unavailable.
- Test case failures.
- Scan interruptions.

Mitigation:

- Validate environment setup before testing.
 - Maintain setup documentation.
-

Security Risks

False Positives

Security tools may incorrectly identify vulnerabilities.

Impact:

- Incorrect reporting.
- Wasted remediation effort.

Mitigation:

- Manually review findings.
 - Validate vulnerabilities before reporting.
-

False Negatives

Some vulnerabilities may not be detected.

Impact:

- Security weaknesses remain unidentified.

Mitigation:

- Use multiple testing approaches.
 - Combine automated and manual validation.
-

Project Risks

Time Constraints

Security testing and documentation activities may require more time than planned.

Impact:

- Delayed project completion.

Mitigation:

- Create realistic schedules.
 - Prioritize critical activities.
-

Knowledge Gaps

Insufficient understanding of security concepts may affect testing quality.

Impact:

- Incorrect vulnerability analysis.
- Incomplete reporting.

Mitigation:

- Study OWASP guidelines.
 - Practice using DVWA and OWASP ZAP before execution.
-

Risk Assessment Summary

Risk	Probability	Impact	Risk Level
OWASP ZAP Connectivity Failure	Medium	High	High
Environment Configuration Issues	High	High	High
Tool Integration Issues	Medium	Medium	Medium
False Positives	Medium	Medium	Medium
False Negatives	Medium	High	High
Project Delays	Medium	Medium	Medium
Knowledge Gaps	Low	Medium	Low

Requirement analysis provides a clear understanding of project objectives, testing boundaries, security expectations, and potential risks. This information serves as the foundation for creating an effective test plan and designing appropriate security test cases for the DVWA security testing automation framework.